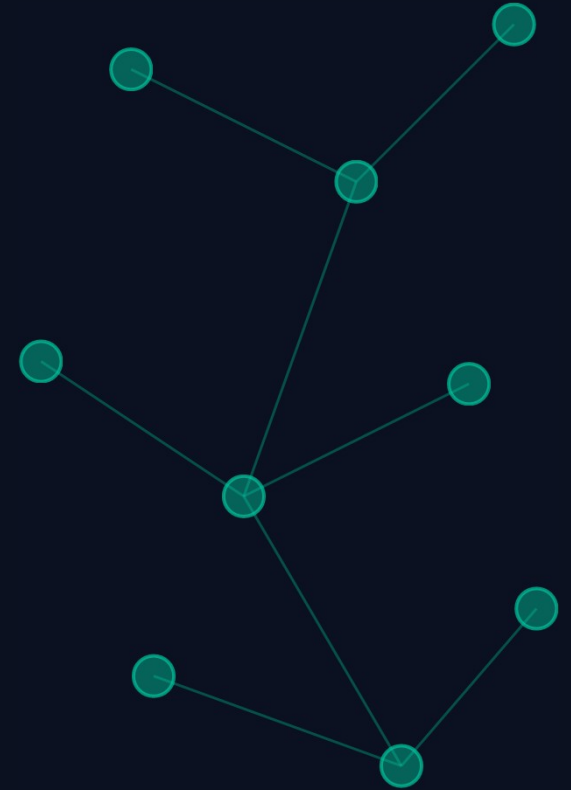


OSINT-ARGUS

Multi-Agent OSINT Protection System

Sprint 1 Review · May 2026

LangGraph · ChromaDB · WHOIS · DNS · LLM Orchestration



01 Sprint Goal Recap

Build the foundation of a working Multi-Agent OSINT pipeline



LangGraph Setup

OrchestratorAgent + DomainAgent in a StateGraph



Tools

Working recon tools callable by agents



Input Classifier

Routes domain / email / URL inputs correctly



ChromaDB + Model Eval

Vector store connected, LLM model selected

02 Project Structure

```
MultiAgent-OSINT-Argus/
├── app/
│   ├── main.py           # Entry point – reads user input, builds initial state, invokes graph
│   ├── graph.py          # LangGraph StateGraph definition – nodes, edges, entry point
│   ├── state.py          # ArgusState TypedDict – shared state schema for all agents
│   └── config.py         # Loads .env – API keys, base URL, model name
├── agents/
│   ├── base_agent.py     # Abstract BaseAgent (ABC) – defines run(state) interface
│   ├── orchestrator_agent.py # LLM-based routing supervisor – sets next_agent
│   └── domain_agent.py   # Full domain analysis via 6 OSINT tools + LLM report
├── models/
│   ├── llm.py            # ChatOpenAI factory – returns configured LLM instance
│   └── router.py         # RouteDecision Pydantic model – structured output schema
├── memory/
│   └── chroma_memory.py  # ChromaDB client – save_analysis() + search_memory()
├── tools/
│   ├── classifier.py     # Regex classifier – fast pre-routing before LLM
│   ├── whois_tool.py     # WHOIS lookup – registrar, dates, name servers
│   ├── dns_tool.py       # DNS records – A, MX, NS
│   └── domain_tools.py   # 4 advanced tools: SSL, SPF/DMARC/DKIM, URLhaus, crt.sh
├── .env                  # Local secrets (not committed)
├── .env.example          # Template for required environment variables
├── requirements.txt       # Python dependencies
└── README.md             # This file
```

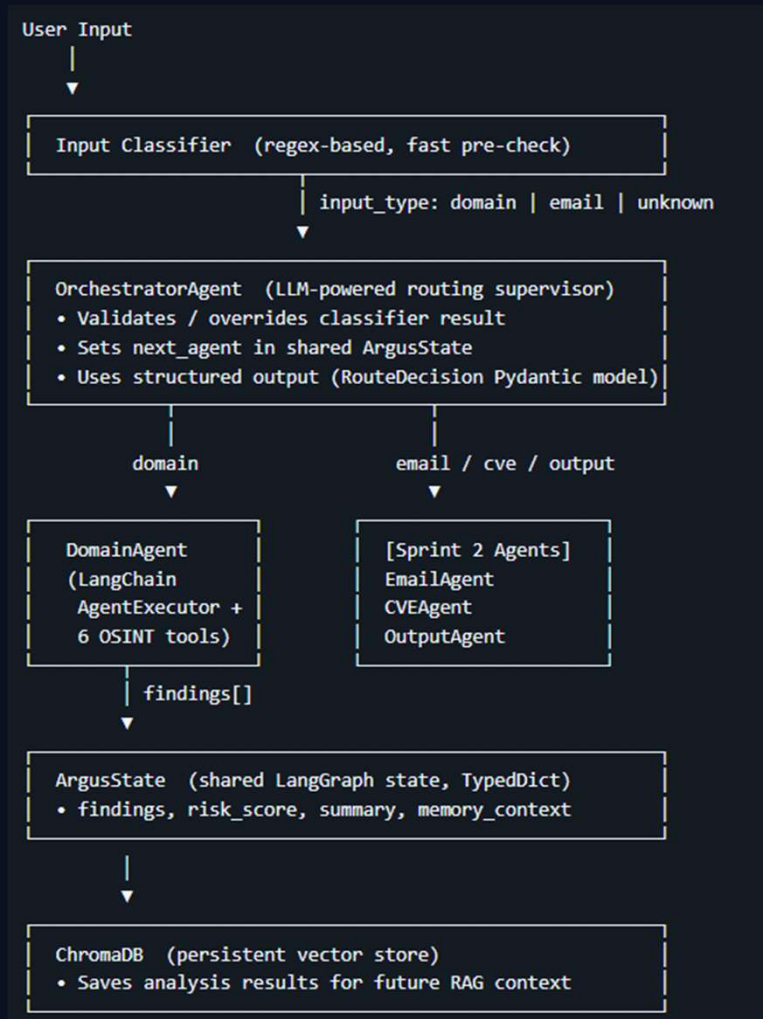
Modular & Structured

Features a strict separation of concerns, cleanly isolating workflow execution

Built for Seamless Expansion

The abstract agent foundation and flexible graph layout make it effortless to scale the system

03 Architecture



DONE

Regex Classifier as LLM Fallback



DONE

Orchestrator output control with pydantic RouteDecision



DONE

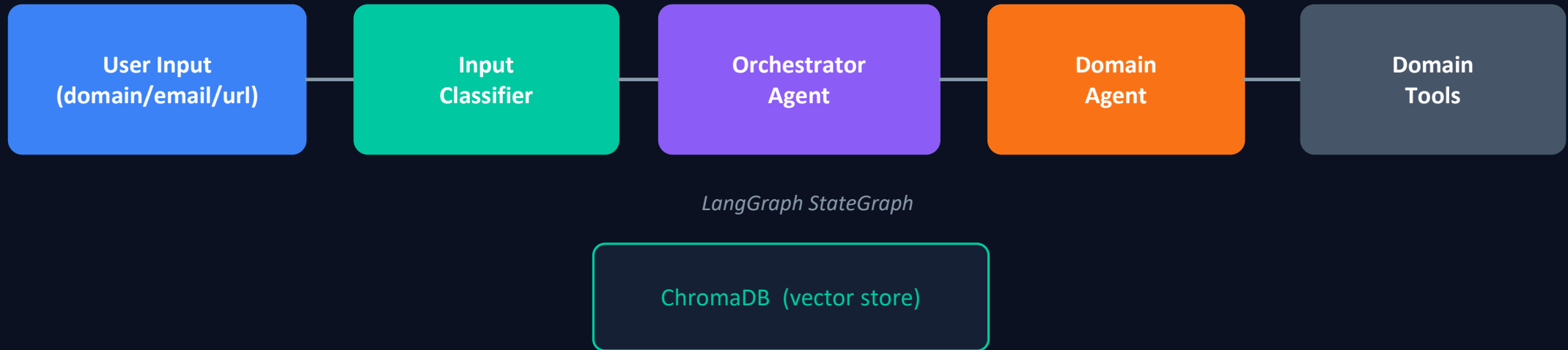
at the moment sequential Tool calling



DONE

ArgusState as shared State

04 Demo & Progress



- ✓ **DONE** StateGraph initialised, agent nodes wired up
- ✓ **DONE** DomainTools (WHOIS lookup, DNS resolution, SSL check, Spoofing check, crtsh, urlhaus)
- ✓ **DONE** Classifier correctly routes all 3 input types
- ✓ **DONE** ChromaDB connected, SAIA LLM API call

05 Learnings

Harder than expected

- LangGraph Setup with Routing Decision
- Finding right Libraries
- Tool implementation

✓ Easier / Positive surprises

- LangGraph state handling.
- LLM prompts (so far).
- Input classifier accuracy was surprisingly high.

06 Sprint 2 Plan

Goal: Full multi-agent system — all 3 input scenarios routed correctly

EmailAgent (main part -> focus)

- Email parsing
- Phishing detection via LLM
- VirusTotal API integration

CVEAgent

- NVD NIST API lookup
- MITRE ATT&CK mapping
- Severity enrichment

OutputAgent

- Risk score aggregation
- Structured report generation
- Human-readable summary

Supervisor Routing

- Test all 3 input paths end-to-end
- Routing edge cases + fallbacks
- Scalable code architecture

OSINT-ARGUS

Sprint 1 complete. Foundation is solid.

Next: EmailAgent · CVEAgent · OutputAgent · Full routing

Questions & Discussion